Practical – 5

Part A — Practice Programs (Hour 1)
Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home.

1. Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total, tracking the largest — one loop handling every shape through polymorphism.
2. Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.
3. (For Advanced Learners – home) Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself.

Sol)

1)

Shapearea.java

```
abstract class Shape {
   abstract double area();
}

class Circle extends Shape {
   double radius;

   Circle(double radius) {
      this.radius = radius;
   }

   double area() {
      return Math.PI * radius * radius;
   }
}

class Rectangle extends Shape {
   double length, width;

   Rectangle(double length, double width) {
```

```
        this.length = length;
        this.width = width;
    }

    double area() {
        return length * width;
    }
}

class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    double area() {
        return 0.5 * base * height;
    }
}

public class ShapeAreas {
    public static void main(String[] args) {

        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6),
            new Triangle(8, 5),
            new Circle(3)
        };

        double total = 0;
        double largest = 0;

        for (Shape s : shapes) {
            double a = s.area();

            System.out.println("Area = " + a);
```

```
            total += a;

            if (a > largest) {
               largest = a;
            }
        }

        System.out.println("Total Area = " + total);
        System.out.println("Largest Area = " + largest);
    }
}
```

Output

```
Area = 78.53981633974483
Area = 24.0
Area = 20.0
Area = 28.274333882308138
Total Area = 150.81415022205296
Largest Area = 78.53981633974483
```

Learning Outcome

- Learned how to create and use an **abstract class** in Java.
- Understood **method overriding** using `area()`.
- Learned how **inheritance** is implemented using `extends`.
- Understood **runtime polymorphism** using a `Shape[]` array.
- Learned to process different objects using a **single loop**.
- Learned to calculate total and largest values using polymorphic objects.

2)

Payroll.java

```
abstract class Employee {

    String name;

    int id;



    Employee(String name, int id) {
```

```
        this.name = name;
        this.id = id;
    }

    abstract double monthlySalary();
}

class FullTime extends Employee {
    double salary;

    FullTime(String name, int id, double salary) {
        super(name, id);
        this.salary = salary;
    }

    double monthlySalary() {
        return salary;
    }
}

class PartTime extends Employee {
    int hours;
    double rate;

    PartTime(String name, int id, int hours, double rate) {
```

```
        super(name, id);
        this.hours = hours;
        this.rate = rate;
    }

    double monthlySalary() {
        return hours * rate;
    }
}

class Intern extends Employee {
    double stipend;

    Intern(String name, int id, double stipend) {
        super(name, id);
        this.stipend = stipend;
    }

    double monthlySalary() {
        return stipend;
    }
}

public class Payroll {
    public static void main(String[] args) {
```

```
Employee[] employees = {
    new FullTime("Kavya", 101, 50000),
    new PartTime("Rahul", 102, 80, 300),
    new Intern("Amit", 103, 10000),
    new FullTime("Priya", 104, 60000)
};

double total = 0;

for (Employee e : employees) {

    double salary = e.monthlySalary();

    System.out.println(
        "Name: " + e.name +
        ", ID: " + e.id +
        ", Salary: " + salary
    );

    if (e instanceof Intern) {
        System.out.println("Note: This employee is an Intern.");
    }

    total += salary;
```

```
        }


        System.out.println("----------------------");
        System.out.println("Total Payroll = " + total);
    }
}
```

Output

Name: Kavya, ID: 101, Salary: 50000.0

Name: Rahul, ID: 102, Salary: 24000.0

Name: Amit, ID: 103, Salary: 10000.0

Note: This employee is an Intern.

Name: Priya, ID: 104, Salary: 60000.0

Total Payroll = 144000.0

Expected Output

Learned to create an **abstract Employee class**.
Understood how different employee types can inherit from the same parent class.
Learned to use **constructors and `super()`** to initialize common data.
Implemented **method overriding** for different salary calculations.
Learned to use **polymorphism** with an `Employee[]` array.
Understood the use of **`instanceof`** to identify a specific subclass.

Learned to calculate the total salary using a single loop.